Movedrop L2

Security Assessment

May 23rd, 2025 — Prepared by OtterSec

Andreas Mantzoutas andreas@osec.io

Robert Chen r@osec.io

Table of Contents

Executive Summary **2**

Overview 2

Key Findings 2

Scope **3**

Findings **4**

Vulnerabilities **5**

OS-MVD-ADV-00 | Merkle Proof Replay 6

OS-MVD-ADV-01 | Flawed Merkle Proof Verification Logic 7

OS-MVD-ADV-02 | Missing Admin Validation Locks Funds Permanently 8

OS-MVD-ADV-03 | Event Spoofing via Resource Index Collision 9

General Findings **10**

OS-MVD-SUG-00 | Inefficient Claim Tracking Mechanism 11

OS-MVD-SUG-01 | Lack of Domain Separation 12

Appendices

Vulnerability Rating Scale **13**

Procedure **14**

© 2025 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

Movement engaged OtterSec to assess the `movedrop` programs. This assessment was conducted between May 6th and May 7th, 2025. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 6 findings throughout this audit engagement.

In particular, we identified a critical vulnerability where the same Merkle proof can be reused multiple times by providing different leaf indices, leading to multiple claims and extracting more than the intended allocation (OS-MVD-ADV-00). Additionally, we highlighted an inconsistency between the result expected by `verify_proof` and the actual return value of `compare_bytes`, resulting in incorrect proof verification (OS-MVD-ADV-01).

We also made recommendations to improve the codebase's efficiency and security hardening. Specifically, we advised against using a vector to track claims, as each claim requires full deserialization, resulting in increased transaction costs (OS-MVD-SUG-00). Lastly, we proposed the incorporation of domain separators to mitigate second preimage attacks against the Merkle tree, particularly if future changes allow leaf nodes to reach 64 bytes in length (OS-MVD-SUG-01).

© 2025 Otter Audits LLC. All Rights Reserved.

02 — Scope

The source code was delivered to us in a git repository at https://github.com/movementlabsxyz/movedrop-l2. This audit was performed against commit d6b1a1d.

A brief description of the programs is as follows:

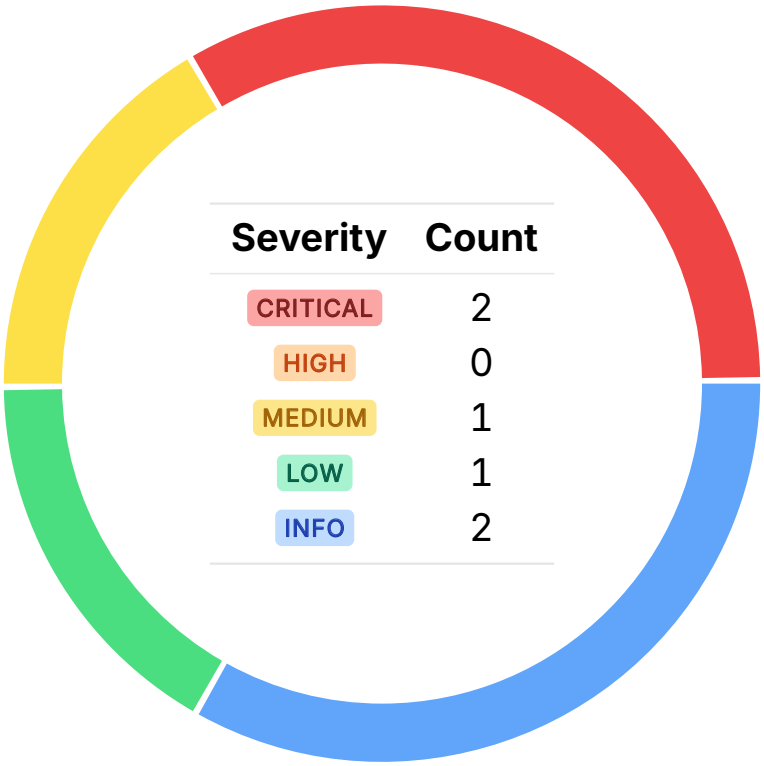
Name	Description
movedrop-l2	The movedrop-l2 module enables token distributions via resource accounts managed by an admin. The direct airdrop supports batch transfers to explicit recipient lists, while the Merkle airdrop allows users to claim tokens by proving inclusion in a Merkle tree, optimizing for gas efficiency.

© 2025 Otter Audits LLC. All Rights Reserved.

03 — Findings

Overall, we reported 6 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	2
HIGH	0
MEDIUM	1
LOW	1
INFO	2

© 2025 Otter Audits LLC. All Rights Reserved.

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-MVD-ADV-00	CRITICAL	RESOLVED	Merkle proof reuse allows for multiple claims.
OS-MVD-ADV-01	CRITICAL	RESOLVED	Flawed comparison logic leads to incorrect proof verification in `verify_proof`.
OS-MVD-ADV-02	MEDIUM	RESOLVED	Missing admin validation in `initialize_airdrop` allows unauthorized initializations and potential fund loss.
OS-MVD-ADV-03	LOW	RESOLVED	The `AirdropEvent` lacks a unique identifier, making it vulnerable to event spoofing.

© 2025 Otter Audits LLC. All Rights Reserved.

Merkle Proof Replay CRITICAL OS-MVD-ADV-00

Description

`claim` allows airdrop beneficiaries to claim their allocated amounts. Users provide their claim amount, along with the proof and the leaf index of their node. The function then constructs the leaf's hash using the caller's address and the amount, validates the proof, and, if successful, transfers the amount to the user while marking the corresponding leaf index as claimed to prevent double claims.

>_ *sources/merkle_airdrop.move* RUST

```rust
public entry fun claim(
    recipient: &signer,
    amount: u64,
    proof: vector<vector<u8>>,
    leaf_index: u64
) acquires AirdropData {
    ...
    let leaf = hash_leaf(recipient_addr, amount);

    // Verify the Merkle proof
    assert!(verify_proof(leaf, proof, airdrop_data.merkle_root),
        error::invalid_argument(EINVALID_PROOF));
    ...
    aptos_account::transfer(&resource_signer, recipient_addr, amount);
    ...
    }
```

However, the issue arises because the leaf index is neither validated against the provided leaf node nor included in the hash or proof verification. This oversight allows the same valid Merkle proof to be reused with different leaf indices, enabling multiple unauthorized claims from a single valid proof.

Remediation

Include the leaf index as part of the node's hash during its construction.

Patch

Fixed in 78782bd.

© 2025 Otter Audits LLC. All Rights Reserved.

Flawed Merkle Proof Verification Logic CRITICAL OS-MVD-ADV-01

Description

`compare_bytes` is responsible for determining the relative ordering of two byte sequences. It evaluates whether the first byte sequence is less than, equal to, or greater than the second, returning 0, 1, or 2, respectively. The `verify_proof` function, responsible for validating Merkle proofs, relies heavily on `compare_bytes`. First, it uses it to enforce the correct ordering of leaf nodes during hashing, and finally to validate that the computed hash matches the expected Merkle root.

>_ *sources/merkle_airdrop.move* RUST

```rust
fun verify_proof(leaf: vector<u8>, proof: vector<vector<u8>>, root: vector<u8>): bool {
    ...
    while (i < len) {
        ...
        let is_left = compare_bytes(&computed_hash, &proof_element) < 0;
        ...
    };
    compare_bytes(&computed_hash, &root) == 0
}
```

However, since `compare_bytes` returns `0` if the first argument is smaller, `1` if both are equal, and `2` if the second is smaller, the `is_left` condition always evaluates to false, leading to incorrect hash computation. Additionally, the function mistakenly returns true when the computed hash is merely less than the Merkle root (i.e., when the comparison yields 0), rather than strictly equal. This flaw completely disrupts the verification logic, enabling unauthorized claims while potentially invalidating legitimate ones.

Remediation

Update the conditions to properly validate the ordering logic and the final hash comparison

Patch

Fixed in 78782bd.

© 2025 Otter Audits LLC. All Rights Reserved.

Missing Admin Validation Locks Funds Permanently MEDIUM OS-MVD-ADV-02

Description

`initialize_airdrop` allows any user to initiate and fund an airdrop. It creates a new resource account derived from the signer's address and a fixed seed, then stores the airdrop's resources and locks the specified amount within that account.

>_ sources/merkle_airdrop.move RUST

```rust
public entry fun initialize_airdrop(
    admin: &signer,
    ...
) {
    let admin_addr = signer::address_of(admin);
    ...
    let (resource_signer, signer_cap) = account::create_resource_account(admin, seed);
    let resource_addr = signer::address_of(&resource_signer);
    ...
    aptos_account::transfer(admin, resource_addr, total_amount);
}
```

However, subsequent functions attempt to access the airdrop's resources using a specific resource account address derived from the fixed seed and the `@movedrop` address, not the initiator's address. This mismatch causes all operations to interact with the pre-determined resource account, ignoring any user-specific instances. Consequently, if a user initializes an airdrop for their own address, the resources are effectively locked and inaccessible, resulting in a permanent loss of funds.

Remediation

Enforce strict access control in `initialize_airdrop` by ensuring only the admin can initialize new airdrops.

Patch

Fixed in 78782bd.

© 2025 Otter Audits LLC. All Rights Reserved.

Event Spoofing via Resource Index Collision LOW OS-MVD-ADV-03

Description

The `direct_airdrop` module allows anyone to create and distribute an airdrop. During distribution, the specified initialization amounts are transferred to users' wallets, and an `AirdropEvent` is emitted, containing the airdrop's details along with a `resource_index` to differentiate it from others.

>_ sources/merkle_airdrop.move RUST

```rust
#[event]
struct AirdropEvent has drop, store {
    resource_index: u64,
    num_recipients: u64,
    total_amount: u64,
    timestamp: u64
}
```

However, the `resource_index` is not uniquely enforced across different airdrops, allowing it to be reused. This results in `AirdropEvent` logs that lack unique identifiers per airdrop, enabling malicious actors to mimic the event details of legitimate airdrops, causing confusion in off-chain indexers and analytics platforms.

Remediation

Introduce a unique identifier in `AirdropEvent` to prevent event collision and ensure reliable tracking.

Patch

Fixed in 5fdd39c.

© 2025 Otter Audits LLC. All Rights Reserved.

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-MVD-SUG-00	The bitmap-based claim tracking mechanism requires full deserialization on every claim, making large-scale airdrops inefficient and costly.
OS-MVD-SUG-01	The lack of domain separation could make the Merkle tree susceptible to second preimage attacks.

© 2025 Otter Audits LLC. All Rights Reserved.

Inefficient Claim Tracking Mechanism

OS-MVD-SUG-00

Description

The current implementation tracks claims using a bitmap to prevent double-claiming. This bitmap is represented as a `vector<u8>`, where each bit corresponds to the claim status of a recipient.

>_ *sources/direct_airdrop.move* RUST

```rust
struct AirdropData has key {
    admin: address,
    merkle_root: vector<u8>,
    claimed_bitmap: vector<u8>,
    ...
}
```

While this design is space-efficient for on-chain storage, it introduces significant performance bottlenecks. Each invocation of the `claim` function requires deserializing the entire vector just to update a single bit. As a result, gas costs scale linearly with the size of the airdrop, making large-scale distributions increasingly inefficient and costly.

Remediation

Replace the `vector<u8>` bitmap with a `Table` representation. This avoids full deserialization and enables sparse storage.

Patch

Fixed in 78782bd.

© 2025 Otter Audits LLC. All Rights Reserved.

Lack of Domain Separation

OS-MVD-SUG-01

Description

The current implementation of the Merkle tree lacks domain separation and uses the same hashing algorithm for both leaf and intermediate nodes. This design flaw allows leaf nodes and internal nodes to be indistinguishable during hash computation. If the leaf nodes are exactly 64 bytes—the output size of typical hash functions—the structure becomes vulnerable to second preimage attacks.

A second preimage attack occurs when an attacker is able to craft a different input that produces the same hash as an existing one. In this context, an attacker could create a 64-byte leaf that hashes identically to an internal node, potentially forging valid Merkle proofs and bypassing verification checks.

Remediation

Introduce domain separation by adding unique prefixes for leaf and intermediate nodes to prevent second preimage collisions.

Patch

Fixed in 78782bd.

© 2025 Otter Audits LLC. All Rights Reserved.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2025 Otter Audits LLC. All Rights Reserved.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2025 Otter Audits LLC. All Rights Reserved.